

Universitat de Girona

Intelligent Field and Robotic Systems Master

Lab1 - Camera Calibration

Prepared by:

Davina Sanghera - u1100402

Rihab Laroussi - u1100330

October 14, 2024

1. Introduction

This lab aims to grasp the basic concepts of camera projection geometry through calibrating a simulated camera. We explore how realworld 3D points project onto a 2D image plane using intrinsic and extrinsic camera parameters. By creating a camera projection matrix, projecting 3D points, and assessing calibration accuracy under various conditions, we gain insights into camera systems' behavior.

The lab is divided into several key steps. In steps 1&2, we define the camera's intrinsic and extrinsic parameters to create the camera projection matrix. In step 3,4 & 5, a set of 3D points is generated and projected onto the 2D image plane using this matrix. In step 6 & 7, the camera projection matrix is estimated using the Hall method based on these 3D points and their 2D projections. In step 8&9, noise is then introduced to the 2D points to assess how it affects the calibration accuracy. Finally, in the last step, the effect of increasing the number of 3D points on the projection error is examined, demonstrating how more data points improve the accuracy of the projection matrix.

2. Implementation

Step 1: Define Camera Parameters

The intrinsic and extrinsic parameters of the camera were provided as follows:

```
%-----STEP 1-----%

% Intrinsic parameters
au = 557.0943; av = 712.9824;
u0 = 326.3819; v0 = 298.6679;

% Location of the world frame in camera coordinates (in mm)
Tx = 100; Ty = 0; Tz = 1500;

% World rotation w.r.t camera coordinates
% Euler XYZ angles
Phix = 0.8*pi/2;
Phiy = -1.8*pi/2;
Phix1 = pi/5;
```

Step 2: Camera Intrinsic and Extrinsic Matrices

From the parameters above, the intrinsic matrix K and the extrinsic transformation cTw were defined. These matrices are used to compute the camera projection matrix P .

```
%-----STEP 2-----%

% Intrinsic matrix K
K = [au 0 u0; 0 av v0; 0 0 1];

% Rotation matrix cRw
rot_x = [1 0 0; 0 cos(Phix) -sin(Phix); 0 sin(Phix) cos(Phix)];
rot_y = [cos(Phiy) 0 sin(Phiy); 0 1 0; -sin(Phiy) 0 cos(Phiy)];
rot_x1 = [1 0 0; 0 cos(Phix1) -sin(Phix1); 0 sin(Phix1) cos(Phix1)];
cRw = rot_x * rot_y * rot_x1;

% Translation matrix ctw
ctw = [100; 0; 1500];

% Transformation matrix cTw
cTw = [cRw ctw; 0 0 0 1];
```

The camera projection matrix P is defined as:

```
33
34 % Projection matrix
35 P = K * [1 0 0 0; 0 1 0 0; 0 0 1 0] * cTw
36
```

Command Window

New to MATLAB? See resources for [Getting Started](#).

```
>> run("Lab1.m")
```

```
P =
```

```
1.0e+05 *
```

```
-0.0050    0.0009   -0.0040    5.4528
-0.0018    0.0074    0.0015    4.4800
 0.0000    0.0000   -0.0000    0.0150
```

Step 3: Define 3D Points

Since the generation of random 3D points is a repeated task within the lab, we defined the function 'generate_hom_3d_points' which randomly generates n homogeneous 3D points within a given range.

```
function [pts_hom_3d] = generate_hom_3d_pts(n, lower, upper)
% Randomly generate a set of n homogeneous 3d points within the range (lower, upper)
pts_cart_3d = randi([lower, upper], n, 3);
pts_hom_3d = [pts_cart_3d, ones(n, 1)];
end
```

This step requires a set of 3D points randomly in the range [-480,480] [480,480] for each axis (X, Y, Z). The set must contain at least 6 non-co-linear and non-coplanar points. Therefore, we pass these parameters to the function to generate the set of 3D points 'pts_3d':

```
%-----STEP 3-----%

% Generate set of 6 3D points
pts_3d = generate_hom_3d_pts(6, -480, 480);
```

Step 4: Project 3D Points onto the Image Plane

Using the projection matrix P , the 3D points were projected onto the image plane.

Again, since this functionality will be reused in further steps, we defined the function 'project_3d_to_image_plane'.

```
function [pts_3d_proj] = project_3d_to_image_plane(pts_3d, P)
% Multiply each 3D point in pts_3d by P
n = length(pts_3d);
pts_3d_proj = [];
for i = 1:n
    pt_proj = P * pts_3d(i, :)' ;
    pts_3d_proj = [pts_3d_proj; pt_proj'];
end
end
```

This function takes the set of 3d points, transposes them so that they can be multiplied by the projection matrix, and then transposes them back. The set of projected points is returned as 'pts_3d_proj'. This function is called from the main program:

```
%-----STEP 4-----%

% Project 3D points onto image plane
pts_3d_proj = project_3d_to_image_plane(pts_3d, P);
```

The projected points were then normalized by dividing by their third coordinate. The function 'convert_to_2d' along with the function call is given below.

```
function [pts_2d] = convert_to_2d(pts_3d_proj)
% Convert to 2D
n = length(pts_3d_proj);
pts_2d = [];
for i = 1:n
    pt_proj = pts_3d_proj(i, :)' ;
    pt_2d = pt_proj(1:2) ./ pt_proj(3);
    pts_2d = [pts_2d; pt_2d'];
end
end
```

```
% Convert to 2D  
pts_2d = convert_to_2d(pts_3d_proj);
```

Step 5: Visualization of the Projected 2D Points

This step helps visualize how the 3D world points appear on the image plane after projection. The following window displays the projected 2D points.

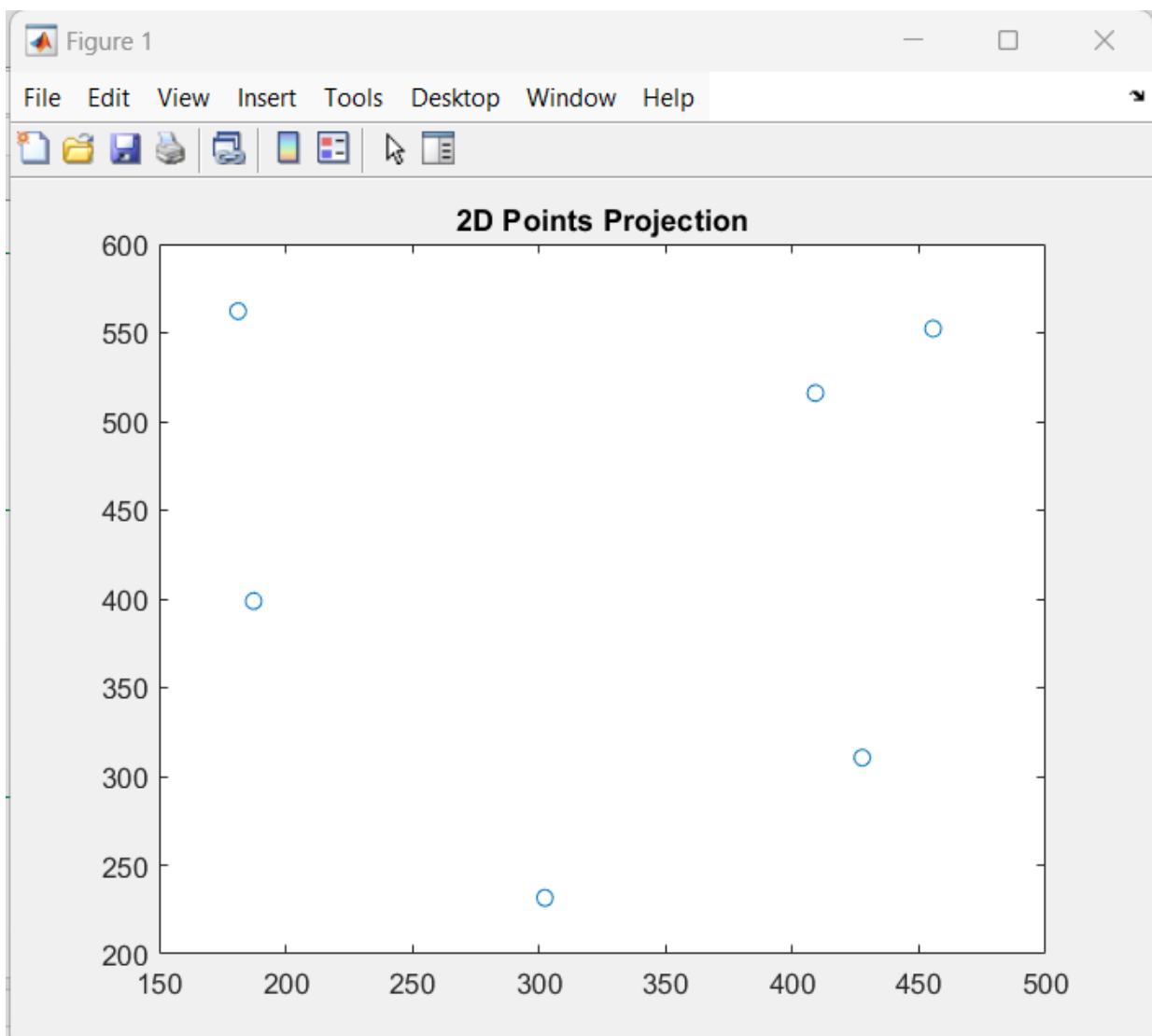


Figure1: The projected 2D points.

Q1: Will the distribution of points in the image affect the accuracy in the computation?

Usually, the distribution of points can impact the accuracy of the computation; having well distributed points decreases the chances of numerical errors during calibration. In cases with noise, clustering can magnify errors, but since our case lacks noise, the distribution has minimal impact on accuracy.

Step 6: Calibration using the Hall Method

Using the known 3D points and their 2D projections, the 3x4 camera projection matrix P is estimated via the Hall method, which we implemented in the function 'hall_method'.

```
function [A] = hall_method(pts_2d, pts_3d)
% Hall method to compute P
Q = [];
B = [];
n = length(pts_2d);

for i = 1:n
    Q1 = [pts_3d(i, 1), pts_3d(i, 2), pts_3d(i, 3), 1, 0, 0, 0, 0, -pts_2d(i, 1)*pts_3d(i, 1), -pts_2d(i, 1)*pts_3d(i, 2), -pts_2d(i, 1)*pts_3d(i, 3)];
    Q2 = [0, 0, 0, 0, pts_3d(i, 1), pts_3d(i, 2), pts_3d(i, 3), 1, -pts_2d(i, 2)*pts_3d(i, 1), -pts_2d(i, 2)*pts_3d(i, 2), -pts_2d(i, 2)*pts_3d(i, 3)];
    Q = [Q; Q1; Q2];
    B1 = pts_2d(i, 1);
    B2 = pts_2d(i, 2);
    B = [B; B1; B2];
end

% Obtain projection matrix
A_vector = Q\B;
A = [A_vector(1), A_vector(2), A_vector(3), A_vector(4); A_vector(5), A_vector(6), A_vector(7), A_vector(8); A_vector(9), A_vector(10), A_vector(11), 1];
end
```

Step 7: Comparison of Matrices and Extraction of Parameters

To compare the projection matrix obtained from the Hall method (A) with the initially defined matrix (P), P must be normalized by its scale factor. Since no noise has been introduced the projection matrices are identical as expected.

```

56
57 %-----STEP 6-----%
58
59 % Hall method to compute P|
60 A = hall_method(pts_2d, pts_3d)
61
62 % Normalise P by its scale factor
63 P_normalised = P/P(3,4)
64

```

Command Window

New to MATLAB? See resources for [Getting Started](#).

```

A =

    -0.3324    0.0624   -0.2662   363.5215
    -0.1207    0.4903    0.1028   298.6679
     0.0001    0.0004   -0.0005    1.0000

P_normalised =

    -0.3324    0.0624   -0.2662   363.5215
    -0.1207    0.4903    0.1028   298.6679
     0.0001    0.0004   -0.0005    1.0000

```

Furthermore, the intrinsic parameter matrix K and the rotation matrix cR_w are extracted from the projection matrix.

```

64
65 %-----STEP 7-----%
66
67 % Extract intrinsic parameters
68 [K, cRw] = get_intrinsics_from_proj_matrix(P)
69

```

Command Window

New to MATLAB? See resources for [Getting Started](#).

```

K =

    557.0943    -0.0000   326.3819
         0   712.9824   298.6679
         0         0    1.0000

cRw =

    -0.9511   -0.1816   -0.2500
    -0.2939    0.7817    0.5501
     0.0955    0.5967   -0.7968

```


We can conclude that the projection matrix from the Hall method matched the initial matrix after normalization. Since no noise was added, the calibration accurately recovered the original matrix, and the camera parameters remained identical.

Step 8: Adding Gaussian Noise

Using the function 'add_Gaussian_noise', Gaussian noise is added to the 2D points, with a variance that ensures 95% of the points have discrepancies in the range $[-1, +1]$ pixels.

```
function [pts_2d_noisy] = add_Gaussian_noise(pts_2d)
% Add Gaussian noise to 2D points
[m, n] = size(pts_2d);
pts_2d_noisy = zeros(m, n);
for i = 1:m
    x_noise = normrnd(0, 1/1.96);
    y_noise = normrnd(0, 1/1.96);
    pts_2d_noisy(i, 1) = pts_2d(i, 1) + x_noise;
    pts_2d_noisy(i, 2) = pts_2d(i, 2) + y_noise;
end
end
```

The projection matrix is recalculated for these noisy points using the 'hall_method' function and the intrinsic and extrinsic parameters are extracted again:

```

69
70 %-----STEP 8-----%
71
72 % Add Gaussian noise to 2D points
73 pts_2d_noisy = add_Gaussian_noise(pts_2d);
74
75 % Hall method to compute noisy P
76 A_noisy = hall_method(pts_2d_noisy, pts_3d);
77
78 % Extract intrinsic parameters
79 [K_noisy, cRw_noisy] = get_intrinsics_from_proj_matrix(A_noisy)
80

```

Command Window

New to MATLAB? See resources for [Getting Started](#).

```

K_noisy =

    559.8018    7.7893   360.7590
         0   728.7512   311.0793
         0         0    1.0000

cRw_noisy =

   -0.9526   -0.2246   -0.2054
   -0.3027    0.7696    0.5622
    0.0318    0.5977   -0.8011

```

Q2: How did the intrinsic parameters change?

The intrinsic parameters, particularly α_u , α_v , and u_0 , v_0 , were slightly altered due to the noise, but remained within a small deviation range.

Q3: Why is the axis skew parameter γ different from zero?

The skew parameter γ becomes non-zero in practical scenarios due to the noise added. When noise is introduced, there are misalignments between the image axes, causing the camera's x and y axes to no longer be perfectly perpendicular.

Step 9: Computing the Average Projection Error

The average projection error is computed as the mean Euclidean distance between the 2D points obtained in Step 4 (without noise) and Step 8 (with noise). This was calculated using the function `average_projection_error`:

```

function [avg_proj_error] = average_projection_error(pts_2d, pts_2d_noisy)
% Compute average projection error
n = length(pts_2d);
all_dists = 0;
for i = 1:n
    dist = sqrt((pts_2d(i, 1) - pts_2d_noisy(i, 1))^2 + ((pts_2d(i, 2) - pts_2d_noisy(i, 2))^2));
    all_dists = all_dists + dist;
end
avg_proj_error = all_dists / n;
end

```

After projecting the noisy points onto the image plane and converting them to 2D, we obtained the average projection error for a set of 6 3D points.

```

%-----STEP 9-----%

% Project 3D points onto image plane using noisy P
pts_3d_proj_noisy = project_3d_to_image_plane(pts_3d, A_noisy);

% Convert to 2D
pts_2d_noisy = convert_to_2d(pts_3d_proj_noisy);

% Compute average projection error
avg_proj_error = average_projection_error(pts_2d, pts_2d_noisy)

```

Step 10: Increasing the Number of 3D Points

Using the previously implemented functions, the process is repeated with 10 and 50 3D points to observe the impact of using more points on the projection matrix's accuracy.

For 10 3D points:

```

%-----STEP 10-----%

% Computing the average projection error for 10 points
pts_3d_10 = generate_hom_3d_pts(10, -480, 480);
pts_3d_proj_10 = project_3d_to_image_plane(pts_3d_10, P);
pts_2d_10 = convert_to_2d(pts_3d_proj_10);
pts_2d_noisy_10 = add_Gaussian_noise(pts_2d_10);
A_noisy_10 = hall_method(pts_2d_noisy_10, pts_3d_10);
pts_3d_proj_noisy_10 = project_3d_to_image_plane(pts_3d_10, A_noisy_10);
pts_2d_noisy_10 = convert_to_2d(pts_3d_proj_noisy_10);
avg_proj_error_10 = average_projection_error(pts_2d_10, pts_2d_noisy_10)

```

For 50 3D points:

```
% Computing the average projection error for 50 points
pts_3d_50 = generate_hom_3d_pts(50, -480, 480);
pts_3d_proj_50 = project_3d_to_image_plane(pts_3d_50, P);
pts_2d_50 = convert_to_2d(pts_3d_proj_50);
pts_2d_noisy_50 = add_Gaussian_noise(pts_2d_50);
A_noisy_50 = hall_method(pts_2d_noisy_50, pts_3d_50);
pts_3d_proj_noisy_50 = project_3d_to_image_plane(pts_3d_50, A_noisy_50);
pts_2d_noisy_50 = convert_to_2d(pts_3d_proj_noisy_50);
avg_proj_error_50 = average_projection_error(pts_2d_50, pts_2d_noisy_50)
```

The average projection error is plotted as a function of the number of points:

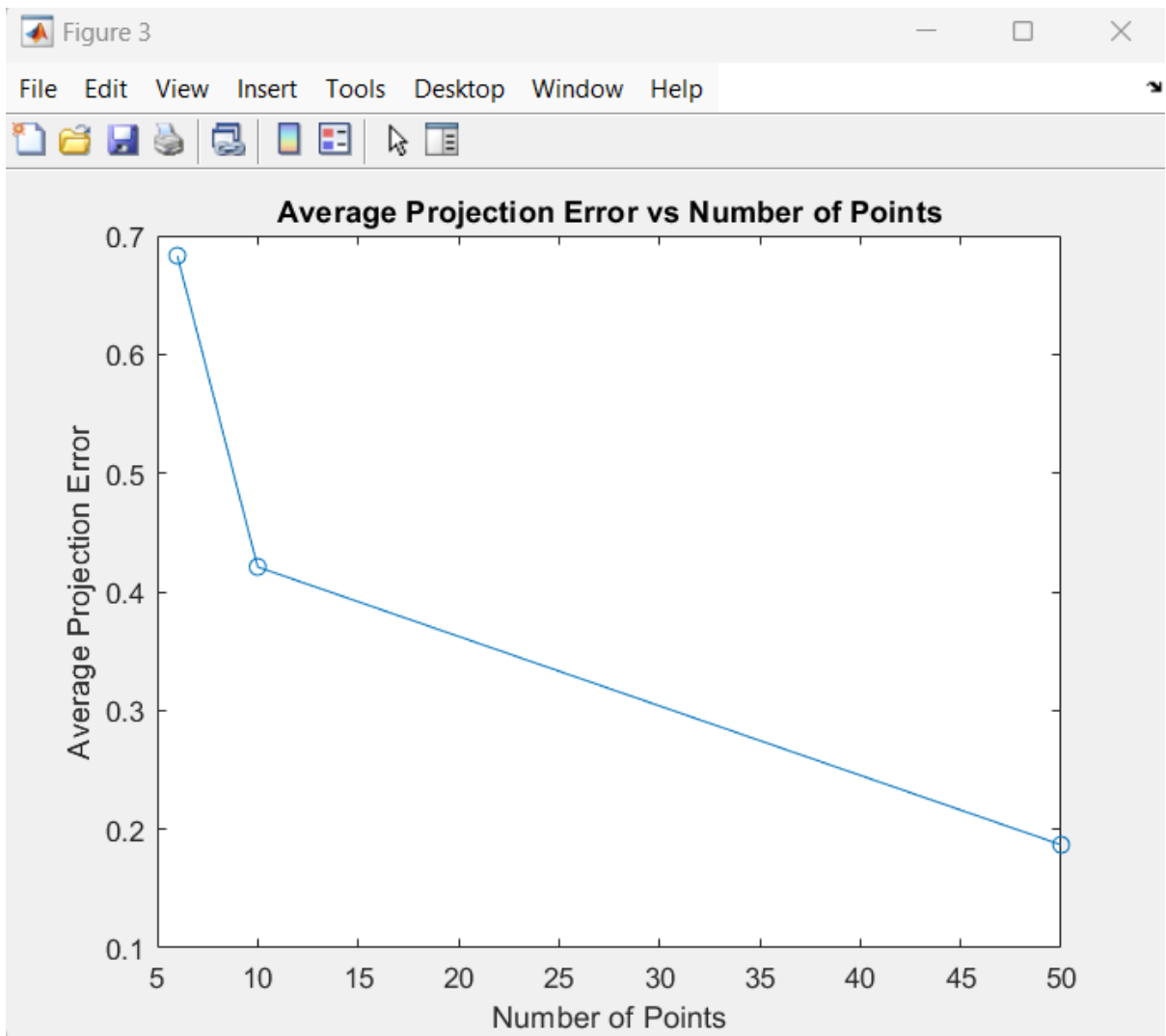


Figure2: The average projection error.

3. Conclusions

The results of the lab indicated that adding more points decreased the average projection error, demonstrating that a higher number of points enhances the accuracy of the projection matrix estimation. Furthermore, noise impacts the intrinsic parameters and introduces slight inaccuracies into the calibration process, particularly affecting the axis skew parameter, γ .